

reduce/flat

An AgentMPI collective scaling analysis

Abstract

The flat reduction is the “manager summarises all the worker reports” pattern: every rank sends its contribution to the root, which folds them locally. Across 21 measured sizes from $p = 2$ to $p = 32$ it logged exactly $p - 1$ messages, reported that count exactly, recorded one round at every size, and recorded a fold depth of exactly $p - 1$ at every size — all four agreeing with the closed form at 21 of 21, with no remainder path and no red crosses. Those last two numbers are the family’s whole content and they are in tension: flat reduce has the shallowest *network* of any reduction (one round; the root talks to everyone) and the deepest *fold* (all $p - 1$ operator applications execute back-to-back on one rank). That combination makes it the only algorithm in `cost.FORMULAS` whose reduce family has *depth* > *rounds*, and it is where I found a real defect: `cost.predict` computes the operator term as $\min(\text{depth}, \text{rounds})$ applications, so for flat reduce it charges *one* application regardless of p . The predicted time is 34.35 s at $p = 2$ and 34.35 s at $p = 64$, and `best_algorithm("reduce", p, n, objective="time")` consequently returns `flat` at every process count. Archived agent-executed runs refute it directly: flat reduce at $p = 8$ took 197.6 s and 251.6 s of root blocking against the model’s 34.35 s, while removing the clamp predicts 239.7 s. The single most important thing to take away is that the sweep measured the very quantity the time model discards — `fold_depth` = $p - 1$, at all 21 sizes — so the fix requires no new instrumentation.

1 What this family is

The `reduce` collective under the `flat` algorithm, measured at 21 process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- [\[ok\]](#) All 21 measured sizes logged exactly the number of messages the closed form predicts.
- [\[note\]](#) Wall times span 0.012–0.076 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

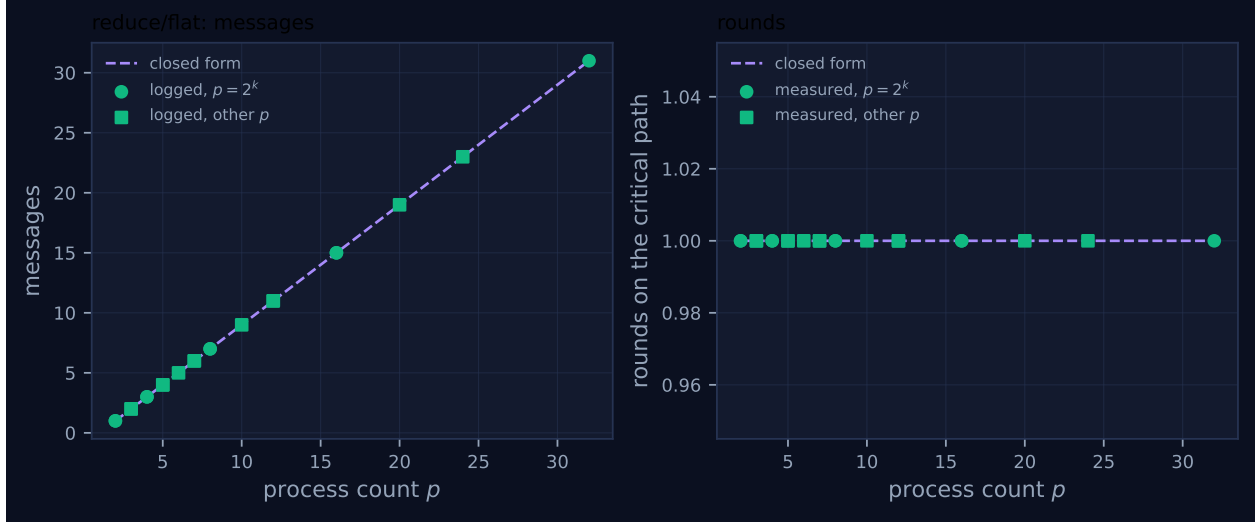


Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective’s own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	mb-free	1	1	1	1	1	0.014	✓	✓
2	mb-smoke	1	1	1	1	1	0.012	✓	✓
3	mb-free	2	2	2	1	1	0.013	✓	✓
3	mb-smoke	2	2	2	1	1	0.013	✓	✓
4	mb-free	3	3	3	1	1	0.015	✓	✓
4	mb-smoke	3	3	3	1	1	0.013	✓	✓
5	mb-free	4	4	4	1	1	0.018	✓	✓
5	mb-smoke	4	4	4	1	1	0.015	✓	✓
6	mb-free	5	5	5	1	1	0.015	✓	✓
7	mb-free	6	6	6	1	1	0.027	✓	✓
7	mb-smoke	6	6	6	1	1	0.040	✓	✓
8	mb-free	7	7	7	1	1	0.016	✓	✓
8	mb-smoke	7	7	7	1	1	0.014	✓	✓
10	mb-free	9	9	9	1	1	0.054	✓	✓
12	mb-free	11	11	11	1	1	0.028	✓	✓
12	mb-smoke	11	11	11	1	1	0.029	✓	✓
16	mb-free	15	15	15	1	1	0.035	✓	✓
16	mb-smoke	15	15	15	1	1	0.047	✓	✓
20	mb-free	19	19	19	1	1	0.073	✓	✓
24	mb-free	23	23	23	1	1	0.067	✓	✓
32	mb-free	31	31	31	1	1	0.076	✓	✓

4 How the algorithm scales

The implementation splits sharply by role. A non-root rank makes one `_csend` of its payload to the root and is finished. The root loops $p - 1$ times over `comm.recv(source=-1)`, accepting arrivals in whatever order they land; if the operator is not commutative it then sorts the contributions

by source rank; and finally it left-folds them, calling `op.fn` once per contribution with an explicit `ReduceContext(depth=i, weight=i+1)`. So the traffic is $p - 1$ messages of n tokens each and the operator is applied $p - 1$ times, all of them on the root. The recorded statistics follow: `rounds = 1` at every rank, and `fold_depth = p - 1` at the root and 0 at every leaf. This is `FORMULAS[("reduce", "flat")]` exactly, which records $(1, p - 1, (p - 1)n, p - 1)$.

The trace shows both halves cleanly. In `mb-free__coll-reduce-flat-8` the seven leaves send between 5.06 and 12.55 ms, in the order 7, 4, 2, 3, 5, 1, 6 — no leaf waits for another, which is what a single round means. The root’s seven `msg.recv` events are then stamped at 15.21, 15.39, 15.50, 15.57, 15.65, 15.71 and 15.78 ms: it drains all seven arrivals in 0.57 ms and records `fold_depth: 7` with 12.1 ms of blocking. With `SUM` as the operator each fold is an integer addition, so the 0.57 ms is the draining and not the folding. That is the crucial caveat about this family’s timings: the sweep measures the network shape of flat reduce faithfully and measures its operator cost not at all, because the operator is free.

Consequently Figure 1’s two panels look nothing alike and both are correct. The left panel is the straight line $p - 1$ with every marker on it, the same line all three of `flat`, `binomial` and `chain` produce. The right panel is a horizontal line at 1: the round count never steps, because there is nothing to step — the depth of a star is one for every p . What the figure cannot show, because the generator’s round axis is the model’s round axis, is the quantity that does grow. Fold depth measured 1, 2, 3, 4, 5, 6, 7, 9, 11, 15, 19, 23, 31 across the thirteen sizes, matching $p - 1$ at all 21 runs. A reader looking only at the figure would conclude that flat reduce is the cheapest reduction available; that conclusion is the defect described below.

The viewer screenshot for `mb-free__coll-reduce-flat-32` shows the fan-in serialising at the root. Rank 0’s lane is the only one carrying a long run of bright message glyphs, spread across the whole 0.07 s span from roughly 0.02 to 0.07 s, while the other thirty-one lanes carry one or two ticks each, clustered early. The contrast with `mb-free__coll-bcast-flat-32`, where rank 0’s glyphs are a *tight* early burst and the other lanes are spread, is exactly the difference between producing $p - 1$ messages as fast as the fabric will take them and consuming $p - 1$ messages as fast as the senders happen to produce them. The same star topology, read in the two directions, has its serialisation in different places.

5 The cost model charges flat reduce one operator application, not $p - 1$

This is a defect in `src/agentmpi/cost.py` rather than in the sweep, and the sweep is what exposes it. `cost.predict` computes the critical-path time as

```
time_s = rounds * (fabric_s + n*beta_in), then time_s += min(op_rounds, rounds) *
message_time(op_cost)
```

where `op_rounds` is the tuple’s fold depth. The comment above it explains the intent: “a reduction additionally pays one operator application per level of the tree”. For `chain` the levels and the applications coincide at $p - 1$; for `binomial` they coincide at $\lceil \log_2 p \rceil$; for `kary` at $\lceil \log_k p \rceil$. For `flat` they do not coincide at all — one level, $p - 1$ applications — and the min takes the smaller. Enumerating `FORMULAS`, `reduce/flat` and `reduce_scatter/linear` are the only two entries in the whole table with `depth > rounds`, so this is the only place in the reduce family where the clamp bites, and it bites completely.

The result is a predicted time that does not depend on p . With default `CostParams` ($\alpha = 12\text{ s}$, $\beta^{-1} = 45\text{ tokens/s}$), $n = 1000\text{ tokens}$ and an operator output of 1000 tokens, `predict("reduce", "flat", p, ...)` returns 34.35 s at $p = 2, 4, 8, 16, 32$ and 64 alike, while `chain` rises from 34.35 to 2,164 s and `binomial` from 34.35 to 206 s over the same range. So `best_algorithm("reduce", p, n, objective="time")` returns `flat` at every process count I tested (4, 8, 16, 32, 64) — recommending, as the latency-optimal reduction, the algorithm whose own docstring says “all $p - 1$ applications execute on the root, so the root’s cost is $(p - 1)$ agent calls” and whose modelled fidelity at $p = 64$ is 0.0395.

Three further pieces of evidence say this is an oversight and not a convention. First, `predict` is internally inconsistent about it: the same function charges the *price* of $(p - 1)$ operator applications, via `message_price(volume, (p-1)*op_cost_tokens)`, while charging the *time* of one. Second, the sibling function `predict_kary` performs the same calculation without a clamp, using `depth * message_time(...)` directly. Third, the paper’s own `tab:reduce-alg` lists `flat`’s applications as “ $p - 1$ (all at root)”, and its evaluation section asserts that “Flat reduction is again slowest-but-one despite needing one round, for the reason the cost model gives: its applications serialise at the root” — a reason the code as written does not in fact give.

Archived measurement settles it. Four runs in this repository execute the reduce family with a real agent-implemented operator at $p = 8$; two of them use `flat`. In `sat-mb-fidelity__fidelity-flat-f-k4` the root’s `coll.reduce` records 197.616 s of blocking over seven `agent.call` events averaging 28.2 s each; in `inc-mb-fidelity-inc__fidelity-flat-f-k4-inc` it records 251.627 s over seven calls averaging 35.9 s. The clamped model predicts 34.35 s — low by factors of 5.8 and 7.3. Evaluating the same formula with the clamp removed, so that all `depth = p - 1` applications are charged, gives 239.68 s, which sits between the two measurements. The number the corrected formula needs is already in the trace and already in the `Prediction` object: `fold_depth`, measured as $p - 1$ at 21 of 21 sizes in this sweep.

Nothing in the test suite pins this down. `test_message_counts_match_cost_formulas` checks message counts and not times; `test_fold_depth_is_logarithmic_for_trees` parametrises over `chain` and `binomial` only, so the one algorithm whose depth exceeds its round count is the one algorithm whose depth the suite never asserts. That is a plausible account of how the clamp survived.

6 Powers of two and everything else

There is no remainder path. The root’s loop is `for _ in range(p - 1)` over `recv(source=-1)`, which is indifferent to the structure of p , and a leaf’s behaviour is a single send. The code executed at $p = 7$ is the code executed at $p = 8$ with one fewer iteration.

The measurements are consistent. All eight non-power-of-two sizes logged $p - 1$ messages (2 at $p = 3$, 4 at $p = 5$, 5 at $p = 6$, 6 at $p = 7$, 9 at $p = 10$, 11 at $p = 12$, 19 at $p = 20$, 23 at $p = 24$), self-reported the same figure, recorded one round, and recorded fold depth $p - 1$. Both marker classes in Figure 1 lie on the closed form in both panels and there are no red crosses.

Token accounting is exact at all 21 sizes, which distinguishes this algorithm from its two siblings. A leaf sends `payload` bare, so the `msg.send` events sum to $p - 1$ tokens and the collective reports $p - 1$. `reduce/binomial` and `reduce/chain` wrap the accumulator in a metadata envelope and charge only the accumulator, logging $20(p - 1)$ and $16(p - 1)$ tokens respectively while reporting $p - 1$; `flat` has no envelope and therefore no discrepancy.

One code path deserves explicit mention because it is `flat reduce`’s unique capability and the sweep

does not exercise it. When `op.commutative` is false, the root sorts the contributions by source rank before folding, so flat reduce evaluates in exact rank order for *any* root. `binomial` cannot: its virtual-rank ordering coincides with communicator rank order only when the root is 0, and the implementation raises `AmpiUsageError` rather than reorder silently. So flat is one of only two legal choices (with `chain`) for a non-commutative operator at a non-zero root. All 21 runs in this sweep used SUM, which is commutative, with `root = 0`, so the sort is never taken and this claim rests on reading the code and on `test_reduce_preserves_order_for_noncommutative_op`, which checks CONCAT against the reference fold for `flat` across nine sizes.

Eight sizes were measured twice. Every count agrees at all eight; the run wall times differ by factors from 1.01 to 1.47.

7 When to choose this algorithm

Flat reduce is the right choice under one condition and the wrong choice by default, and the two numbers this sweep measured are exactly the two sides of that.

The condition is that only the root has the judgement to combine, and that $p - 1$ contributions fit in its context. Under those constraints there is no alternative: any tree delegates a fold to an interior rank, which is precisely what a harness with one authoritative integrator does not want. Flat also has the network property that no tree has — a single round, so the last contribution to arrive determines when the fold can start and no contribution waits behind another in flight. Measured, that shows as $p - 1$ leaf sends finishing before the root begins draining, and 12.1 ms of root blocking at $p = 8$ against `chain`’s 92.1 ms.

The reason it is nevertheless the wrong default is that its $p - 1$ applications are serial on one rank, and for a semantic operator an application is an agent invocation. At $p = 8$ with a real operator, archived runs give flat 197.6 and 251.6 s of root blocking against `binomial`’s 77.6 and 80.3 s and `kary(k=4)`’s 53.1 and 51.8 s — so flat is 2.5 to 3.2 times slower than the binary tree and roughly four times slower than the 4-ary one, despite needing one round instead of three or two. It beats only `chain`. That ordering is the point of the whole reduce comparison: all three of `flat`, `binomial` and `chain` perform $p - 1$ applications and send $p - 1$ messages, so they differ only in how those applications are arranged, and arranging them all on one rank is the worst arrangement for latency even though it is the best for network depth.

On fidelity the intuition and this repository’s own measurement part company, and it would be wrong to claim otherwise. Structurally, flat’s fold depth of $p - 1$ is the worst available: the cost model’s $F(d) = (1 - \delta)^d$ puts flat at 0.204 against `binomial`’s 0.774 at $p = 32$ with $\delta = 0.05$, and the mechanism is real — the root’s accumulator is re-encoded $p - 1$ times, and the archived reduce traces show that each such re-encoding produces a genuinely new artifact. But the repository’s fidelity experiment measured retention at 0.3854 under *every* algorithm at $p = 8$ (`inc-mb-fidelity-inc`, 37 of 96 items surviving at fold depth 7, 3 and 2 alike) and 1.0000 under every algorithm when the budget was not binding (`sat-mb-fidelity`). Under a uniform enforced output budget the last application is the binding one, so depth buys nothing in quality either way. The defensible case against flat reduce is therefore cost, not fidelity: it is the second-slowest and the second-most-expensive in output tokens of the four measured algorithms, and it earns that by serialising work a tree would have run concurrently.

8 Threats to this reading

These runs contain no agents, and for this family that is the threat rather than a caveat. Every one of the 21 runs reports `executors = {"none": p}`, zero `agent.call` events, zero busy time, an idle fraction of 1.0 and \$0.000 of cost. The operator was SUM with a scalar payload, so each of the $p - 1$ applications the root performs is an integer addition costing nothing measurable — the whole quantity that makes flat reduce expensive is set to zero by construction. The sweep therefore validates the *counts* ($p - 1$ messages, 1 round, fold depth $p - 1$, at 21 of 21) and can say nothing about the duration. The agent-executed figures I quote for the flat-versus-tree comparison come from `sat-mb-fidelity`, `inc-mb-fidelity-inc` and `fid-synth`, which are $p = 8$ and $p = 16$ single points rather than a sweep, and two of the four use a surrogate function operator rather than a model.

The wall times at large p do not measure the collective either. In `mb-free_coll-reduce-flat-32` the thirty-two `rank.init` events span 2.1 to 63.3 ms while the run lasts 76.3 ms, so most of the run is ranks starting up and the 0.076 s in the generated table is a launch measurement. The maximum per-rank blocking inside the collective (49.2 ms at $p = 32$) is the only duration I would defend, and it is a handful of SQLite transactions.

The defect argument depends on how one reads the model’s intent, and I should be explicit about the weakest link. If `rounds` were meant to be an upper bound on concurrent-plus-serial critical path, and `fold_depth` were meant purely as a fidelity statistic with no time consequence, then the clamp would be intentional and the flat-reduce time would be a deliberate statement that a star costs one round. I do not believe that reading survives the three internal inconsistencies — the price term charging $p - 1$, `predict_kary` not clamping, and the paper’s table listing $p - 1$ applications — but it is the reading that would make me wrong, and no comment in `cost.py` settles it either way. What is not in doubt is the consequence: `best_algorithm` under the time objective currently recommends `flat` at every p , and the two archived agent-executed flat runs are 5.8 and 7.3 times slower than it predicts.

Finally, some configurations are unexercised. All 21 runs used `root = 0` and a commutative operator, so the source-sorting branch that gives flat reduce its unique order-preservation guarantee is never taken. All 21 used a one-token payload, so the volume term $(p - 1)n$ is validated only at $n = 1$, and the context pressure that is flat reduce’s real constraint — $p - 1$ contributions in one rank’s window — is not exercised at all: zero of 1,110 receptions across the six sweeps admitted a single token.